

Specialized Optimization Methods

CONTENTS

12.1	Nonlinear Least-Squares	227
12.1.1	Gauss-Newton	228
12.1.2	Levenberg-Marquardt	229
12.2	Iteratively Reweighted Least-Squares	230
12.3	Coordinate Descent and Alternation	231
12.3.1	Identifying Candidates for Alternation	231
12.3.2	Augmented Lagrangians and ADMM	235
12.4	Global Optimization	240
12.4.1	Graduated Optimization	241
12.4.2	Randomized Global Optimization	243
12.5	Online Optimization	244

OPTIMIZATION algorithms like Newton's method are completely generic approaches to minimizing a function $f(\vec{x})$, with or without constraints on $\vec{x}$. These algorithms make few assumptions about the form of f or the constraints. Contrastingly, by designing the conjugate gradient algorithm specifically for minimizing the objective $f(\vec{x}) \equiv \frac{1}{2}\vec{x}^\top A\vec{x} - \vec{b}^\top \vec{x} + c$, we were able to guarantee more reliable and efficient behavior than general algorithms.

In this chapter, we continue to exploit special structure to solve optimization problems, this time for more complex nonlinear objectives. Replacing monolithic generic algorithms with ones tailored to a given problem can make optimization faster and easier to troubleshoot, although doing so requires more implementation effort than calling a pre-packaged solver.

12.1 NONLINEAR LEAST-SQUARES

Recall the nonlinear regression problem posed in Example 9.1. If we wish to fit a function $y = ce^{ax}$ to a set of data points $(x_1, y_1), \dots, (x_k, y_k)$, an optimization mimicking linear least-squares is to minimize the function

$$E(a, c) \equiv \sum_i (y_i - ce^{ax_i})^2.$$

This energy reflects the fact that we wish $y_i - ce^{ax_i} \approx 0$ for all i .

More generally, suppose we are given a set of functions $f_1(\vec{x}), \dots, f_k(\vec{x})$ for $\vec{x} \in \mathbb{R}^n$. If we want $f_i(\vec{x}) \approx 0$ for all i , then a reasonable objective trading off between these terms is

$$E_{\text{NLS}}(\vec{x}) \equiv \frac{1}{2} \sum_i [f_i(\vec{x})]^2.$$

Objective functions of this form are known as *nonlinear least-squares* problems. For the exponential regression problem above, we would take $f_i(a, c) \equiv y_i - ce^{ax_i}$.

12.1.1 Gauss-Newton

When we run Newton's method to minimize a function $f(\vec{x})$, we must know the gradient *and* Hessian of f . Knowing only the gradient of f is not enough, since approximating functions with planes provides no information about their extrema. The BFGS algorithm carries out optimization without Hessians, but its approximate Hessians depend on the sequence of iterations and hence are not local to the current iterate.

Contrastingly, the Gauss-Newton algorithm for nonlinear least-squares makes the observation that approximating each f_i with a linear function yields a nontrivial curved approximation of E_{NLS} since each term in the sum is squared. The main feature of this approach is that it requires only *first-order* approximation of the f_i 's rather than Hessians.

Suppose we write

$$f_i(\vec{x}) \approx f_i(\vec{x}_0) + [\nabla f_i(\vec{x}_0)] \cdot (\vec{x} - \vec{x}_0).$$

Then, we can approximate E_{NLS} with E_{NLS}^0 given by

$$E_{\text{NLS}}^0(\vec{x}) = \frac{1}{2} \sum_i (f_i(\vec{x}_0) + [\nabla f_i(\vec{x}_0)] \cdot (\vec{x} - \vec{x}_0))^2.$$

Define $F(\vec{x}) \equiv (f_1(\vec{x}), f_2(\vec{x}), \dots, f_k(\vec{x}))$ by stacking the f_i 's into a column vector. Then,

$$E_{\text{NLS}}^0(\vec{x}) = \frac{1}{2} \|F(\vec{x}_0) + DF(\vec{x}_0)(\vec{x} - \vec{x}_0)\|_2^2,$$

where DF is the Jacobian of F . Minimizing $E_{\text{NLS}}^0(\vec{x})$ is a *linear* least-squares problem $-F(\vec{x}_0) \approx DF(\vec{x}_0)(\vec{x} - \vec{x}_0)$ that can be solved via the normal equations:

$$\vec{x} = \vec{x}_0 - (DF(\vec{x}_0)^\top DF(\vec{x}_0))^{-1} DF(\vec{x}_0)^\top F(\vec{x}_0).$$

More practically, as we have discussed, the system can be solved using the QR factorization of $DF(\vec{x}_0)$ or—in higher dimensions—using conjugate gradients and related methods.

We can view $\vec{x}$ from minimizing $E_{\text{NLS}}^0(\vec{x})$ as an improved approximation of the minimum of $E_{\text{NLS}}(\vec{x})$ starting from $\vec{x}_0$. The Gauss-Newton algorithm iterates this formula to solve nonlinear least-squares:

$$\vec{x}_{k+1} = \vec{x}_k - (DF(\vec{x}_k)^\top DF(\vec{x}_k))^{-1} DF(\vec{x}_k)^\top F(\vec{x}_k).$$

This iteration is not guaranteed to converge in all situations. Given an initial guess sufficiently close to the minimum of the nonlinear least-squares problem, however, the approximation above behaves similarly to Newton's method and even can have quadratic convergence. Given the nature of the Gauss-Newton approximation, the algorithm works best when the optimal objective value $E_{\text{NLS}}(\vec{x}^*)$ is small; convergence can suffer when the optimal value is relatively large.

12.1.2 Levenberg-Marquardt

The Gauss-Newton algorithm uses an approximation $E_{\text{NLS}}^0(\vec{x})$ of the nonlinear least-squares energy as a proxy for $E_{\text{NLS}}(\vec{x})$ that is easier to minimize. In practice, this approximation is likely to fail as $\vec{x}$ moves farther from $\vec{x}_0$, so we might modify the Gauss-Newton step to include a step size limitation:

$$\begin{aligned} & \min_{\vec{x}} E_{\text{NLS}}^0(\vec{x}) \\ & \text{subject to } \|\vec{x} - \vec{x}_0\|_2^2 \leq \Delta. \end{aligned}$$

That is, we now restrict our change in $\vec{x}$ to have norm less than some user-provided value Δ ; the Δ neighborhood about $\vec{x}_0$ is called a *trust region*. Denote $H \equiv DF(\vec{x}_0)^\top DF(\vec{x}_0)$ and $\delta\vec{x} \equiv \vec{x} - \vec{x}_0$. Then, we can solve:

$$\begin{aligned} & \min_{\delta\vec{x}} \frac{1}{2} \delta\vec{x}^\top H \delta\vec{x} + F(\vec{x}_0)^\top DF(\vec{x}_0) \delta\vec{x} \\ & \text{subject to } \|\delta\vec{x}\|_2^2 \leq \Delta. \end{aligned}$$

That is, we displace $\vec{x}$ by minimizing the Gauss-Newton approximation after imposing the step size restriction. This problem has the following KKT conditions (see §10.2.2):

$$\begin{aligned} & \text{Stationarity: } \vec{0} = H\delta\vec{x} + DF(\vec{x}_0)^\top F(\vec{x}_0) + 2\mu\delta\vec{x} \\ & \text{Primal feasibility: } \|\delta\vec{x}\|_2^2 \leq \Delta \\ & \text{Complementary slackness: } \mu(\Delta - \|\delta\vec{x}\|_2^2) = 0 \\ & \text{Dual feasibility: } \mu \geq 0. \end{aligned}$$

Define $\lambda \equiv 2\mu$. Then, the stationarity condition can be written as follows:

$$(H + \lambda I_{n \times n})\delta\vec{x} = -DF(\vec{x}_0)^\top F(\vec{x}_0).$$

Assume the constraint $\|\delta\vec{x}\|_2^2 \leq \Delta$ is active, that is, $\|\delta\vec{x}\|_2^2 = \Delta$. Then, except in degenerate cases $\lambda > 0$; combining this inequality with the fact that H is positive semidefinite, $H + \lambda I_{n \times n}$ must be positive definite.

The Levenberg-Marquardt algorithm *starts* from this stationarity formula, taking the following step derived from a user-supplied parameter $\lambda > 0$ [82, 85]:

$$\vec{x} = \vec{x}_0 - (DF(\vec{x}_0)^\top DF(\vec{x}_0) + \lambda I_{n \times n})^{-1} DF(\vec{x}_0)^\top F(\vec{x}_0).$$

This linear system also can be derived by applying Tikhonov regularization to the Gauss-Newton linear system. When λ is small, it behaves similarly to the Gauss-Newton algorithm, while large λ results in a gradient descent step for E_{NLS} .

Rather than specifying Δ as introduced above, Levenberg-Marquardt steps fix $\lambda > 0$ directly. By the KKT conditions, *a posteriori* we know this choice corresponds to having taken $\Delta = \|\vec{x} - \vec{x}_0\|_2^2$. As $\lambda \rightarrow \infty$, the step from Levenberg-Marquardt satisfies $\|\vec{x} - \vec{x}_0\|_2 \rightarrow 0$; so, we can regard Δ and λ as approximately inversely proportional.

Typical approaches adaptively adjust the *damping parameter* λ during each iteration:

$$\vec{x}_{k+1} = \vec{x}_k - (DF(\vec{x}_k)^\top DF(\vec{x}_k) + \lambda_k I_{n \times n})^{-1} DF(\vec{x}_k)^\top F(\vec{x}_k).$$

For instance, we can scale up λ_k when the step in $E_{\text{NLS}}(\vec{x})$ agrees well with the approximate value predicted by $E_{\text{NLS}}^0(\vec{x})$, since this corresponds to increasing the size of the neighborhood in which the Gauss-Newton approximation is effective.

12.2 ITERATIVELY REWEIGHTED LEAST-SQUARES

Continuing in our consideration of least-squares problems, suppose we wish to minimize a function of the form:

$$E_{\text{IRLS}}(\vec{x}) \equiv \sum_i f_i(\vec{x})[g_i(\vec{x})]^2.$$

We can think of $f_i(\vec{x})$ as a *weight* on the least-squares term $g_i(\vec{x})$.

Example 12.1 (L^p optimization). Similar to the compressed sensing problems in §10.4.1, given $A \in \mathbb{R}^{m \times n}$ and $\vec{b} \in \mathbb{R}^m$ we can generalize least-squares by minimizing

$$E_p(\vec{x}) \equiv \|A\vec{x} - \vec{b}\|_p^p.$$

Choosing $p = 1$ can promote sparsity in the residual $\vec{b} - A\vec{x}$. We can write this function in an alternative form:

$$E_p(\vec{x}) = \sum_i (\vec{a}_i \cdot \vec{x} - b_i)^{p-2} (\vec{a}_i \cdot \vec{x} - b_i)^2.$$

Here, we denote the rows of A as $\vec{a}_i^\top$. Then, $E_p = E_{\text{IRLS}}$ after defining:

$$\begin{aligned} f_i(\vec{x}) &= (\vec{a}_i \cdot \vec{x} - b_i)^{p-2} \\ g_i(\vec{x}) &= \vec{a}_i \cdot \vec{x} - b_i. \end{aligned}$$

The *iteratively reweighted least-squares* (IRLS) algorithm makes use of the following fixed point iteration:

$$\vec{x}_{k+1} = \min_{\vec{x}} \sum_i f_i(\vec{x}_k)[g_i(\vec{x}_{k+1})]^2.$$

In the minimization, $\vec{x}_k$ is fixed, so the optimization is a least-squares problem over the g_i 's. When g_i is linear, the minimization can be carried out via linear least-squares; otherwise we can use the nonlinear least-squares techniques in §12.1.

Example 12.2 (L^1 optimization). Continuing Example 12.1, suppose we take $p = 1$. Then,

$$E_1(\vec{x}) = \sum_i |\vec{a}_i \cdot \vec{x} - b_i| = \sum_i \frac{1}{|\vec{a}_i \cdot \vec{x} - b_i|} (\vec{a}_i \cdot \vec{x} - b_i)^2.$$

This functional leads to the following IRLS iteration, after adjustment for numerical issues:

$w_i \leftarrow [\max(\vec{a}_i \cdot \vec{x} - b_i , \delta)]^{-1}$	▷ Recompute weights
$\vec{x} \leftarrow \min_{\vec{x}} \sum_i w_i (\vec{a}_i \cdot \vec{x} - b_i)^2$	▷ Linear least-squares

The parameter $\delta > 0$ avoids division by zero; large values of δ make better-conditioned linear systems but worse approximations of the original $\|\cdot\|_1$ problem.

Example 12.3 (Weiszfeld algorithm). Recall the geometric median problem from Example 9.3. In this problem, given $\vec{x}_1, \dots, \vec{x}_k \in \mathbb{R}^n$, we wish to minimize

$$E(\vec{x}) \equiv \sum_i \|\vec{x} - \vec{x}_i\|_2.$$

Similar to the L^1 problem in Example 12.2, we can write this function like a weighted least-squares problem:

$$E(\vec{x}) \equiv \sum_i \frac{1}{\|\vec{x} - \vec{x}_i\|_2} \|\vec{x} - \vec{x}_i\|_2^2.$$

Then, IRLS provides the *Weiszfeld algorithm* for geometric median problems:

$w_i \leftarrow [\max(\ \vec{x} - \vec{x}_i\ _2, \delta)]^{-1}$	▷ Recompute weights
$\vec{x} \leftarrow \min_{\vec{x}} \sum_i w_i (\vec{x} - \vec{x}_i)^2$	▷ Linear least-squares

We can solve for the second step of the Weiszfeld algorithm in closed form. Differentiating the objective with respect to $\vec{x}$ shows

$$\vec{0} = \sum_i 2w_i(\vec{x} - \vec{x}_i) \implies \vec{x} = \frac{\sum_i w_i \vec{x}_i}{\sum_i w_i}.$$

Thus, the two alternating steps of Weiszfeld's algorithm can be carried out efficiently as:

$w_i \leftarrow [\max(\ \vec{x} - \vec{x}_i\ _2, \delta)]^{-1}$	▷ Recompute weights
$\vec{x} \leftarrow \frac{\sum_i w_i \vec{x}_i}{\sum_i w_i}$	▷ Weighted centroid

IRLS algorithms are straightforward to formulate, so they are worth trying if an optimization can be written in the form of E_{IRLS} . When g_i is linear for all i as in Example 12.2, each iteration of IRLS can be carried out quickly using Cholesky factorization, QR, conjugate gradients, and so on, avoiding line search and other more generic strategies.

It is difficult to formulate general conditions under which IRLS will reach the minimum of E_{IRLS} . Often, iterates must be approximated somewhat as in the introduction of δ to Example 12.2 to avoid division by zero and other degeneracies. In the case of L^1 optimization, however, IRLS can be shown with small modification to converge to the optimal point [31].

12.3 COORDINATE DESCENT AND ALTERNATION

Suppose we wish to minimize a function $f : \mathbb{R}^{n+m} \rightarrow \mathbb{R}$. Rather than viewing the input as a single variable $\vec{x} \in \mathbb{R}^{n+m}$, we might write f in an alternative form as $f(\vec{x}, \vec{y})$, for $\vec{x} \in \mathbb{R}^n$ and $\vec{y} \in \mathbb{R}^m$. One strategy for optimization is to fix $\vec{y}$ and minimize f with respect to $\vec{x}$, fix $\vec{x}$ and minimize f with respect to $\vec{y}$, and repeat:

for $i \leftarrow 1, 2, \dots$	
$\vec{x}_{i+1} \leftarrow \min_{\vec{x}} f(\vec{x}, \vec{y}_i)$	▷ Optimize $\vec{x}$ with $\vec{y}$ fixed
$\vec{y}_{i+1} \leftarrow \min_{\vec{y}} f(\vec{x}_{i+1}, \vec{y})$	▷ Optimize $\vec{y}$ with $\vec{x}$ fixed

In this *alternating* approach, the value of $f(\vec{x}_i, \vec{y}_i)$ decreases monotonically as i increases since a minimization is carried out at each step. We cannot prove that alternation always reaches a global or even local minimum, but in many cases it can be an efficient option for otherwise challenging problems.

12.3.1 Identifying Candidates for Alternation

There are a few reasons why we might wish to perform alternating optimization:

- The individual problems over $\vec{x}$ and $\vec{y}$ are optimizations in a lower dimension and may converge more quickly.
- We may be able to split the variables in such a way that the individual $\vec{x}$ and $\vec{y}$ steps are far more efficient than optimizing both variables jointly.

Below we provide a few examples of alternating optimization in practice.

Example 12.4 (Generalized PCA). In the PCA problem from §7.2.5, we are given a data matrix $X \in \mathbb{R}^{n \times k}$ whose columns are k data points in $\mathbb{R}^n$. We seek a basis in $\mathbb{R}^n$ of size d such that the projection of the data points onto the basis introduces minimal approximation error; we will store this basis in the columns of $C \in \mathbb{R}^{n \times d}$. Classical PCA minimizes $\|X - CY\|_{\text{Fro}}^2$ over both C and Y , where the columns of $Y \in \mathbb{R}^{d \times k}$ are the coefficients of the data points in the C basis. If C is constrained to be orthogonal, then $Y = C^\top X$, recovering the formula in our previous discussion.

The Frobenius norm in PCA is somewhat arbitrary: The relevant relationship is $X - CY \approx 0$. Alternative PCA models minimize $\mu(X - CY)$ over C and Y , for some other energy function $\mu : \mathbb{R}^{n \times k} \rightarrow \mathbb{R}$ favoring matrices with entries near zero; μ can provide enhanced robustness to noise or encode application-specific assumptions. Taking $\mu(M) \equiv \|M\|_{\text{Fro}}^2$ recovers classical PCA; another popular choice is *robust PCA* [71], which takes $\mu(M) \equiv \sum_{ij} |M_{ij}|$.

The product CY in $\mu(X - CY)$ makes the energy nonlinear and nonconvex. A typical minimization routine for this problem uses alternation: First optimize C with Y fixed, then optimize Y with C fixed, and repeat. Whereas optimizing the energy with respect to C and Y jointly might require a generic large-scale method, the individual alternating C and Y steps can be easier:

- When $\mu(M) = \|M\|_{\text{Fro}}^2$, both the Y and C alternations are least-squares problems, leading to the *alternating least-squares* (ALS) algorithm for classical PCA.
- When $\mu(M) \equiv \sum_{ij} |M_{ij}|$, the Y and C alternations are linear programs, which can be optimized using the techniques mentioned in §10.4.1.

Example 12.5 (ARAP). Recall the planar “as-rigid-as-possible” (ARAP) problem introduced in Example 10.5:

$$\begin{aligned} & \text{minimize}_{R_v, \vec{y}_v} \sum_{v \in V} \sum_{(v,w) \in E} \|R_v(\vec{x}_v - \vec{x}_w) - (\vec{y}_v - \vec{y}_w)\|_2^2 \\ & \text{subject to } R_v^\top R_v = I_{2 \times 2} \quad \forall v \in V \\ & \quad \vec{y}_v \text{ fixed } \forall v \in V_0. \end{aligned}$$

Solving for the matrices $R_v \in \mathbb{R}^{2 \times 2}$ and vertex positions $\vec{y}_v \in \mathbb{R}^2$ simultaneously is a highly nonlinear and nonconvex task, especially given the orthogonality constraint $R_v^\top R_v = I_{2 \times 2}$. There is one $\vec{y}_v$ and one R_v for each vertex v of a triangle mesh with potentially thousands or even millions of vertices, so such a direct optimization using quasi-Newton methods requires a large-scale linear solve per iteration and still is prone to finding local minima.

Instead, [116] suggests alternating between the following two steps:

1. Fixing the R_v matrices and optimizing only for the positions $\vec{y}_v$:

$$\begin{aligned} & \text{minimize}_{\vec{y}_v} \sum_{v \in V} \sum_{(v,w) \in E} \|R_v(\vec{x}_v - \vec{x}_w) - (\vec{y}_v - \vec{y}_w)\|_2^2 \\ & \text{subject to } \vec{y}_v \text{ fixed } \forall v \in V_0. \end{aligned}$$

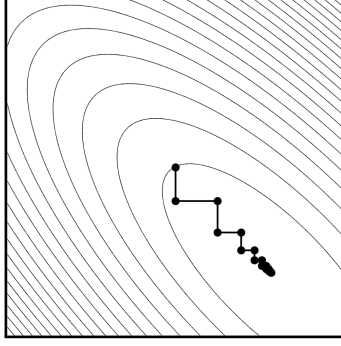


Figure 12.1 Coordinate descent in two dimensions alternates between minimizing in the horizontal and vertical axis directions.

This least-squares problem can be solved using a sparse, positive-definite linear system of equations.

2. Fixing the $\vec{y}_v$'s and optimizing for the R_v 's. No energy terms or constraints couple any pair R_v, R_w for $v, w \in V$, so we can solve for each matrix R_v independently. That is, rather than solving for $4|V|$ unknowns simultaneously, we loop over $v \in V$, solving the following optimization for each $R_v \in \mathbb{R}^{2 \times 2}$:

$$\begin{aligned} & \text{minimize}_{R_v} \sum_{(v,w) \in E} \|R_v(\vec{x}_v - \vec{x}_w) - (\vec{y}_v - \vec{y}_w)\|_2^2 \\ & \text{subject to } R_v^\top R_v = I_{2 \times 2}. \end{aligned}$$

This optimization problem is an instance of the Procrustes problem from §7.2.4 and can be solved in closed-form using a 2×2 SVD. We have replaced a large-scale minimization with the application of a formula that can be evaluated in parallel for each vertex, a massive computational savings.

Alternating between optimizing for the $\vec{y}_v$'s with the R_v 's fixed and vice versa decreases the energy using two efficient pieces of machinery, sparse linear solvers and 2×2 SVD factorization. This can be far more efficient than considering the $\vec{y}_v$'s and R_v 's simultaneously, and in practice a few iterations can be sufficient to generate elastic deformations like the one shown in Figure 10.3. Extensions of ARAP even run in real time, optimizing fast enough to provide interactive feedback to artists editing two- and three-dimensional shapes.

Example 12.6 (Coordinate descent). Taking the philosophy of alternating optimization to an extreme, rather than splitting the inputs of $f : \mathbb{R}^n \rightarrow \mathbb{R}$ into two variables, we could view f as a function of several variables $f(x_1, x_2, \dots, x_n)$. Then, we could cycle through each input x_i , performing a one-dimensional optimization in each step. This lightweight algorithm, illustrated in Figure 12.1, is known as *coordinate descent*.

For instance, suppose we wish to solve the least-squares problem $A\vec{x} \approx \vec{b}$ by minimizing $\|A\vec{x} - \vec{b}\|_2^2$. As in Chapter 11, line search over any single x_i can be solved in closed form. If the columns of A are vectors $\vec{a}_1, \dots, \vec{a}_n$, then as shown in §1.3.1 we can write $A\vec{x} - \vec{b} =$

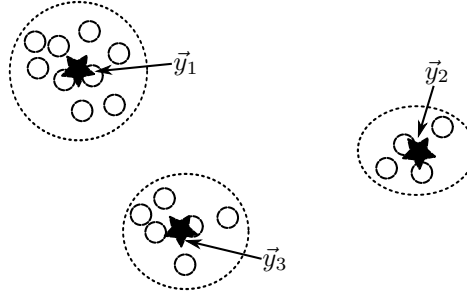


Figure 12.2 The k -means algorithm seeks cluster centers $\vec{y}_i$ that partition a set of data points $\vec{x}_1, \dots, \vec{x}_m$ based on their closest center.

$x_1 \vec{a}_1 + \dots + x_n \vec{a}_n - \vec{b}$. By this expansion,

$$0 = \frac{\partial}{\partial x_i} \|x_1 \vec{a}_1 + \dots + x_n \vec{a}_n - \vec{b}\|_2^2 = 2(A\vec{x} - \vec{b}) \cdot \vec{a}_i = \sum_j \left[\left(\sum_k a_{ji} a_{jk} x_k \right) - a_{ji} b_j \right].$$

Solving this equation for x_i yields the following coordinate descent update for x_i :

$$x_i \leftarrow \frac{\vec{a}_i \cdot \vec{b} - \sum_{k \neq i} x_k (\vec{a}_i \cdot \vec{a}_k)}{\|\vec{a}_i\|_2^2}.$$

Coordinate descent for least-squares iterates this formula over $i = 1, 2, \dots, n$ repeatedly until convergence. This approach has efficient localized updates and appears in machine learning methods where A has many more rows than columns, sampled from a data distribution. We have traded a global method for one that locally updates the solution $\vec{x}$ by solving extremely simple subproblems.

Example 12.7 (k -means clustering). Suppose we are given a set of data points $\vec{x}_1, \dots, \vec{x}_m \in \mathbb{R}^n$ and wish to group these points into k clusters based on distance, as in Figure 12.2. Take $\vec{y}_1, \dots, \vec{y}_k \in \mathbb{R}^n$ to be the centers of clusters $1, \dots, k$, respectively. To cluster the data by assigning each point $\vec{x}_i$ to a single cluster centered at $\vec{y}_c$, the k -means technique optimizes the following energy:

$$E(\vec{y}_1, \dots, \vec{y}_k) \equiv \sum_{i=1}^m \min_{c \in \{1, \dots, k\}} \|\vec{x}_i - \vec{y}_c\|_2^2.$$

In words, E measures the total squared distance of the data points $\vec{x}_i$ to their closest cluster center $\vec{y}_c$.

Define $c_i \equiv \arg \min_{c \in \{1, \dots, k\}} \|\vec{x}_i - \vec{y}_c\|_2^2$; that is, c_i is the index of the cluster center $\vec{y}_{c_i}$ closest to $\vec{x}_i$. Using this substitution, we can write an expanded formulation of the k -means objective as follows:

$$E(\vec{y}_1, \dots, \vec{y}_k; c_1, \dots, c_m) \equiv \sum_{i=1}^m \|\vec{x}_i - \vec{y}_{c_i}\|_2^2.$$

The variables c_i are integers, but we can optimize them jointly with the $\vec{y}$'s using alternation:

- When the c_i 's are fixed, the optimization for the $\vec{y}_j$'s is a least-squares problem whose solution can be written in closed form as

$$\vec{y}_j = \frac{\sum_{c_i=j} \vec{x}_i}{|\{c_i = j\}|}.$$

That is, $\vec{y}_j$ is the average of the points $\vec{x}_i$ assigned to cluster j .

- The optimization for c_i also can be carried out in closed form using the expression $c_i \equiv \arg \min_{c \in \{1, \dots, k\}} \|\vec{x}_i - \vec{y}_c\|_2^2$ by iterating from 1 to k for each i . This iteration just assigns each $\vec{x}_i$ to its closest cluster center.

This alternation is known as the k -means algorithm and is a popular method for clustering. One drawback of this method is that it is sensitive to the initial guesses of $\vec{y}_1, \dots, \vec{y}_k$. In practice, k -means is often run several times with different initial guesses, and only the best output is preserved. Alternatively, methods like “ k -means++” specifically design initial guesses of the $\vec{y}_i$'s to encourage convergence to a better local minimum [3].

12.3.2 Augmented Lagrangians and ADMM

Nonlinear constrained problems are often the most challenging optimization tasks. While the general algorithms in §10.3 are applicable, they can be sensitive to the initial guess of the minimizer, slow to iterate due to large linear solves, and slow to converge in the absence of more information about the problems at hand. Using these methods is easy from an engineering perspective since they require providing only a function and its derivatives, but with some additional work on paper, certain objective functions can be tackled using faster techniques, many of which can be parallelized on multiprocessor machines. It is worth checking if a problem can be solved via one of these strategies, especially when the dimensionality is high or the objective has a number of similar or repeated terms.

In this section, we consider an alternating approach to equality-constrained optimization that has gained considerable attention in recent literature. While it can be used out-of-the-box as yet another generic optimization algorithm, its primary value appears to be in the decomposition of complex minimization problems into simpler steps that can be iterated, often in parallel. In large part we will follow the development of [14], which contains many examples of applications of this class of techniques.

As considered in Chapter 10, the equality-constrained optimization problem can be stated as follows:

$$\begin{aligned} &\text{minimize } f(\vec{x}) \\ &\text{subject to } g(\vec{x}) = \vec{0}. \end{aligned}$$

One incarnation of the barrier method suggested in §10.3.2 optimizes an unconstrained objective with a quadratic penalty:

$$f_\rho(\vec{x}) = f(\vec{x}) + \frac{1}{2}\rho\|g(\vec{x})\|_2^2.$$

As $\rho \rightarrow \infty$, critical points of f_ρ satisfy the $g(\vec{x}) = \vec{0}$ constraint more and more closely. The trade-off for this method, however, is that the optimization becomes poorly conditioned as

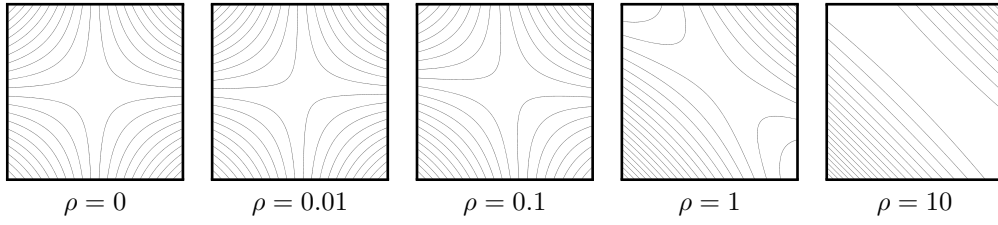


Figure 12.3 We can optimize $f(x, y) \equiv xy$ subject to $x + y = 1$ approximately by minimizing the penalized version $f_\rho(x, y) = xy + \rho(x + y - 1)^2$. As ρ increases, however, level sets of xy get obscured in favor of enforcing the constraint.

ρ becomes large. This effect is illustrated in Figure 12.3; when ρ is large, the level sets of f_ρ mostly are dedicated to enforcing the constraint rather than minimizing the objective $f(\vec{x})$, making it difficult to distinguish between $\vec{x}$'s that all satisfy the constraint.

Alternatively, by the method of Lagrange multipliers (Theorem 1.1), we can seek first-order optima of this problem as the critical points of $\Lambda(\vec{x}, \vec{\lambda})$ given by

$$\Lambda(\vec{x}, \vec{\lambda}) \equiv f(\vec{x}) - \vec{\lambda}^\top g(\vec{x}).$$

This Lagrangian does not suffer from conditioning issues that affect the quadratic penalty method. On the other hand, it replaces a minimization problem—which can be solved by moving “downhill”—with a more challenging saddle point problem in which critical points should be minima of Λ with respect to $\vec{x}$ and maxima of Λ with respect to $\vec{\lambda}$. Optimizing by alternatively minimizing with respect to $\vec{x}$ and maximizing with respect to $\vec{\lambda}$ can be unstable; intuitively this makes some sense since it is unclear whether Λ should be small or large.

The *augmented Lagrangian* method for equality-constrained optimization combines the quadratic penalty and Lagrangian strategies, using the penalty to “soften” individual iterations of the alternation for optimizing Λ described above. It replaces the original equality-constrained optimization problem with the following equivalent augmented problem:

$$\begin{aligned} &\text{minimize } f(\vec{x}) + \frac{1}{2}\rho\|g(\vec{x})\|_2^2 \\ &\text{subject to } g(\vec{x}) = \vec{0}. \end{aligned}$$

Any $\vec{x}$ satisfying the $g(\vec{x}) = \vec{0}$ constraint makes the second objective term vanish. But, when the constraint is not exactly satisfied, the second energy term biases the objective toward points $\vec{x}$ that approximately satisfy the equality constraint. In other words, during iterations of augmented Lagrangian optimization, the $\rho\|g(\vec{x})\|_2^2$ acts like a rubber band pulling $\vec{x}$ closer to the constraint set even during the minimization step.

This modified problem has a new Lagrangian given by

$$\Lambda_\rho(\vec{x}, \vec{\lambda}) \equiv f(\vec{x}) + \frac{1}{2}\rho\|g(\vec{x})\|_2^2 - \vec{\lambda}^\top g(\vec{x}).$$

Hence, the augmented Lagrangian method optimizes this objective by alternating as follows:

for $i \leftarrow 1, 2, \dots$ $\vec{\lambda}_{i+1} \leftarrow \vec{\lambda}_i - \rho g(\vec{x}_i)$ $\vec{x}_i \leftarrow \min_{\vec{x}} \Lambda_\rho(\vec{x}, \vec{\lambda}_{i+1})$	$\triangleright$ Dual update $\triangleright$ Primal update
---	--

The dual update step can be thought of as a gradient ascent step for $\vec{\lambda}$. The parameter ρ here no longer has to approach infinity for exact constraint satisfaction, since the Lagrange multiplier enforces the constraint regardless. Instead, the quadratic penalty serves to make sure the output of the $\vec{x}$ iteration does not violate the constraints too strongly.

Augmented Lagrangian optimization has the advantage that it alternates between applying a formula to update $\vec{\lambda}$ and solving an *unconstrained* minimization problem for $\vec{x}$. For many optimization problems, however, the unconstrained objective still may be non-differentiable or difficult to optimize. A few special cases, e.g., Uzawa iteration for dual decomposition [124], can be effective for optimization but in many circumstances quasi-Newton algorithms outperform this approach with respect to speed and convergence.

A small alteration to general augmented Lagrangian minimization, however, yields the *alternating direction method of multipliers* (ADMM) for optimizing slightly more specific objectives of the form

$$\begin{aligned} & \text{minimize } f(\vec{x}) + h(\vec{z}) \\ & \text{subject to } A\vec{x} + B\vec{z} = \vec{c}. \end{aligned}$$

Here, the optimization variables are both $\vec{x}$ and $\vec{z}$, where $f, h : \mathbb{R}^n \rightarrow \mathbb{R}$ are given functions and the equality constraint is linear. As we will show, this form encapsulates many important optimization problems. We will design an algorithm that carries out alternation between the two primal variables $\vec{x}$ and $\vec{z}$, as well as between primal and dual optimization.

The augmented Lagrangian in this case is

$$\Lambda_\rho(\vec{x}, \vec{z}, \vec{\lambda}) \equiv f(\vec{x}) + h(\vec{z}) + \frac{1}{2}\rho\|A\vec{x} + B\vec{z} - \vec{c}\|_2^2 + \vec{\lambda}^\top (A\vec{x} + B\vec{z} - \vec{c}).$$

Alternating in three steps between optimizing $\vec{x}$, $\vec{z}$, and $\vec{\lambda}$ suggests a modification of the augmented Lagrangian method:

for $i \leftarrow 1, 2, \dots$	
$\vec{x}_{i+1} \leftarrow \arg \min_{\vec{x}} \Lambda_\rho(\vec{x}, \vec{z}_i, \vec{\lambda}_i)$	▷ $\vec{x}$ update
$\vec{z}_{i+1} \leftarrow \arg \min_{\vec{z}} \Lambda_\rho(\vec{x}_{i+1}, \vec{z}, \vec{\lambda}_i)$	▷ $\vec{z}$ update
$\vec{\lambda}_{i+1} \leftarrow \vec{\lambda}_i + \rho(A\vec{x}_{i+1} + B\vec{z}_{i+1} - \vec{c})$	▷ Dual update

In this algorithm, $\vec{x}$ and $\vec{z}$ are optimized one at a time; the augmented Lagrangian method would optimize them jointly. Although this splitting can require more iterations for convergence, clever choices of $\vec{x}$ and $\vec{z}$ lead to powerful division-of-labor strategies for breaking down difficult problems. Each individual iteration will take *far* less time, even though more iterations may be needed for convergence. In a sense, ADMM is a “meta-algorithm” used to design optimization techniques. Rather than calling a generic package to minimize Λ_ρ with respect to $\vec{x}$ and $\vec{z}$, we will find choices of $\vec{x}$ and $\vec{z}$ that make individual steps fast.

Before working out examples of ADMM in action, it is worth noting that it is guaranteed to converge to a critical point of the objective under fairly weak conditions. For instance, ADMM reaches a global minimum when f and h are convex and Λ_ρ has a saddle point. ADMM has also been observed to converge even for nonconvex problems, although current theoretical understanding in this case is limited. In practice, ADMM tends to be quick to generate *approximate* minima of the objective but can require a long tail of iterations to squeeze out the last decimal points of accuracy; for this reason, some systems use ADMM to do initial large-scale steps and transition to other algorithms for localized optimization.

We dedicate the remainder of this section to working out examples of ADMM in practice. The general pattern is to split the optimization variables into $\vec{x}$ and $\vec{z}$ in such a way that

the two primal update steps each can be carried out efficiently, preferably in closed form or decoupling so that parallelized computations can be used to solve many subproblems at once. This makes individual iterations of ADMM inexpensive.

Example 12.8 (Nonnegative least-squares). Suppose we wish to minimize $\|A\vec{x} - \vec{b}\|_2^2$ with respect to $\vec{x}$ subject to the constraint $\vec{x} \geq \vec{0}$. The $\vec{x} \geq \vec{0}$ constraint rules out using Gaussian elimination, but ADMM provides one way to bypass this issue.

Consider solving the following equivalent problem:

$$\begin{aligned} & \text{minimize } \|A\vec{x} - \vec{b}\|_2^2 + h(\vec{z}) \\ & \text{subject to } \vec{x} = \vec{z}. \end{aligned}$$

Here, we define the new function $h(\vec{z})$ as follows:

$$h(\vec{z}) = \begin{cases} 0 & \vec{z} \geq \vec{0} \\ \infty & \text{otherwise.} \end{cases}$$

The function $h(\vec{z})$ is discontinuous, but it is convex. This equivalent form of nonnegative least-squares may be harder to read, but it provides an effective ADMM splitting.

For this optimization, the augmented Lagrangian is

$$\Lambda_\rho(\vec{x}, \vec{z}, \vec{\lambda}) = \|A\vec{x} - \vec{b}\|_2^2 + h(\vec{z}) + \frac{1}{2}\rho\|\vec{x} - \vec{z}\|_2^2 + \vec{\lambda}^\top(\vec{x} - \vec{z}).$$

For fixed $\vec{z}$ with $z_i \neq \infty$ for all i , then Λ_ρ is differentiable with respect to $\vec{x}$. Hence, we can carry out the $\vec{x}$ step of ADMM by setting the gradient with respect to $\vec{x}$ equal to $\vec{0}$:

$$\begin{aligned} \vec{0} &= \nabla_{\vec{x}} \Lambda_\rho(\vec{x}, \vec{z}, \vec{\lambda}) \\ &= 2A^\top A\vec{x} - 2A^\top \vec{b} + \rho(\vec{x} - \vec{z}) + \vec{\lambda} \\ &= (2A^\top A + \rho I_{n \times n})\vec{x} + (\vec{\lambda} - 2A^\top \vec{b} - \rho\vec{z}) \\ \implies \vec{x} &= (2A^\top A + \rho I_{n \times n})^{-1}(2A^\top \vec{b} + \rho\vec{z} - \vec{\lambda}). \end{aligned}$$

This linear solve is a Tikhonov-regularized least-squares problem. For extra speed, the Cholesky factorization of $2A^\top A + \rho I_{n \times n}$ can be computed before commencing ADMM and used to find $\vec{x}$ in each iteration.

Minimizing Λ_ρ with respect to $\vec{z}$ can be carried out in closed form. Any objective function involving h effectively constrains each component of $\vec{z}$ to be nonnegative, so we can find $\vec{z}$ using the following optimization:

$$\begin{aligned} & \text{minimize}_{\vec{z}} \frac{1}{2}\rho\|\vec{x} - \vec{z}\|_2^2 + \vec{\lambda}^\top(\vec{x} - \vec{z}) \\ & \text{subject to } \vec{z} \geq \vec{0}. \end{aligned}$$

The $\|A\vec{x} - \vec{b}\|_2^2$ term in the full objective is removed because it has no $\vec{z}$ dependence. This problem *decouples* over the components of $\vec{z}$ since no energy terms involve more than one dimension of $\vec{z}$ at a time. So, we can solve many instances of the following one-dimensional problem:

$$\begin{aligned} & \text{minimize}_{z_i} \frac{1}{2}\rho(x_i - z_i)^2 + \lambda_i(x_i - z_i) \\ & \text{subject to } z_i \geq 0. \end{aligned}$$

In the absence of the $z_i \geq 0$ constraint, the objective is minimized when $0 = \rho(z_i - x_i) - \lambda_i \implies z_i = x_i + \lambda_i/\rho$; when this value is negative, we fix $z_i = 0$.

Hence, the ADMM algorithm for nonnegative least-squares is:

```

for  $i \leftarrow 1, 2, \dots$ 
   $\vec{x}_{i+1} \leftarrow (2A^\top A + \rho I_{n \times n})^{-1}(2A^\top \vec{b} + \rho \vec{z}_i - \vec{\lambda}_i)$   $\triangleright \vec{x}$  update; least-squares
   $\vec{z}^0 \leftarrow \vec{\lambda}_i / \rho + \vec{x}_{i+1}$   $\triangleright$  Unconstrained  $\vec{z}$  formula
   $\vec{z}_{i+1} \leftarrow \text{ELEMENTWISE-MAX}(\vec{z}^0, \vec{0})$   $\triangleright$  Enforce  $\vec{z} \geq \vec{0}$ 
   $\vec{\lambda}_{i+1} \leftarrow \vec{\lambda}_i + \rho(\vec{x}_{i+1} - \vec{z}_{i+1})$   $\triangleright$  Dual update

```

This algorithm for nonnegative least-squares took our original problem—a quadratic program that could require difficult constrained optimization techniques—and replaced it with an alternation between a linear solve for $\vec{x}$, a formula for $\vec{z}$, and a formula for $\vec{\lambda}$. These individual steps are straightforward to implement and efficient computationally.

Example 12.9 (ADMM for geometric median). Returning to Example 12.3, we can reconsider the energy $E(\vec{x})$ for the geometric median problem using the machinery of ADMM:

$$E(\vec{x}) \equiv \sum_{i=1}^N \|\vec{x} - \vec{x}_i\|_2.$$

This time, we will split the problem into two unknowns $\vec{z}_i, \vec{x}$:

$$\begin{aligned} & \text{minimize } \sum_i \|\vec{z}_i\|_2 \\ & \text{subject to } \vec{z}_i + \vec{x} = \vec{x}_i \quad \forall i. \end{aligned}$$

The augmented Lagrangian for this problem is:

$$\Lambda_\rho = \sum_i \left[\|\vec{z}_i\|_2 + \frac{1}{2}\rho \|\vec{z}_i + \vec{x} - \vec{x}_i\|_2^2 + \vec{\lambda}_i^\top (\vec{z}_i + \vec{x} - \vec{x}_i) \right].$$

As a function of $\vec{x}$, the augmented Lagrangian is differentiable and hence to find the $\vec{x}$ iteration we write:

$$\begin{aligned} \vec{0} &= \nabla_{\vec{x}} \Lambda_\rho = \sum_i \left[\rho(\vec{x} - \vec{x}_i + \vec{z}_i) + \vec{\lambda}_i \right] \\ \Rightarrow \vec{x} &= \frac{1}{N} \sum_i \left[\vec{x}_i - \vec{z}_i - \frac{1}{\rho} \vec{\lambda}_i \right]. \end{aligned}$$

The optimization for the $\vec{z}_i$'s decouples over i when $\vec{x}$ is fixed, so after removing constant terms we minimize $\|\vec{z}_i\|_2 + \frac{1}{2}\rho \|\vec{z}_i + \vec{x} - \vec{x}_i\|_2^2 + \vec{\lambda}_i^\top \vec{z}_i$ for each $\vec{z}_i$ separately. We can combine the second and third terms by “completing the square” as follows:

$$\begin{aligned} \frac{1}{2}\rho \|\vec{z}_i + \vec{x} - \vec{x}_i\|_2^2 + \vec{\lambda}_i^\top \vec{z}_i &= \frac{1}{2}\rho \|\vec{z}_i\|_2^2 + \rho \vec{z}_i^\top \left(\frac{1}{\rho} \vec{\lambda}_i + \vec{x} - \vec{x}_i \right) + \text{const.} \\ &= \frac{1}{2}\rho \left\| \vec{z}_i + \frac{1}{\rho} \vec{\lambda}_i + \vec{x} - \vec{x}_i \right\|_2^2 + \text{const.} \end{aligned}$$

The constant terms can have $\vec{x}$ dependence since it is fixed in the $\vec{z}_i$ iteration. Defining $\vec{z}^0 \equiv -\frac{1}{\rho} \vec{\lambda}_i - \vec{x} + \vec{x}_i$, in the $\vec{z}_i$ iteration we have shown that we can solve:

$$\min_{\vec{z}_i} \left[\|\vec{z}_i\|_2 + \frac{1}{2}\rho \|\vec{z}_i - \vec{z}^0\|_2^2 \right].$$

Written in this form, it is clear that the optimal $\bar{z}_i$ satisfies $\bar{z}_i = t\bar{z}^0$ for some $t \in [0, 1]$, since the two terms of the objective balance the distance of $\bar{z}_i$ to $\vec{0}$ and to $\bar{z}^0$. After dividing by $\|\bar{z}^0\|_2$, we can solve:

$$\min_{t \geq 0} \left[t + \frac{1}{2} \rho \|\bar{z}^0\|_2 (t - 1)^2 \right].$$

Using elementary calculus techniques we find:

$$t = \begin{cases} 1 - 1/\rho \|\bar{z}^0\|_2 & \text{when } \rho \|\bar{z}^0\|_2 \geq 1 \\ 0 & \text{otherwise.} \end{cases}$$

Taking $\bar{z}_i = t\bar{z}^0$ finishes the $\bar{z}$ iteration of ADMM.

In summary, the ADMM algorithm for geometric medians is as follows:

for $i \leftarrow 1, 2, \dots$	
$\vec{x} \leftarrow \frac{1}{N} \sum_i \left[\vec{x}_i - \bar{z}_i - \frac{1}{\rho} \vec{\lambda}_i \right]$	▷ $\vec{x}$ update
for $j \leftarrow 1, 2, \dots, N$	▷ Can parallelize
$\bar{z}^0 \leftarrow -\frac{1}{\rho} \vec{\lambda}_i - \vec{x} + \vec{x}_i$	
$t \leftarrow \begin{cases} 1 - 1/\rho \ \bar{z}^0\ _2 & \text{when } \rho \ \bar{z}^0\ _2 \geq 1 \\ 0 & \text{otherwise} \end{cases}$	
$\bar{z}_j \leftarrow t\bar{z}^0$	▷ $\bar{z}$ update
$\vec{\lambda}_j \leftarrow \vec{\lambda}_j + \rho(\bar{z}_i + \vec{x} - \vec{x}_i)$	▷ Dual update

The examples above show the typical ADMM strategy, in which a difficult nonlinear problem is split into two subproblems that can be carried out in closed form or via more efficient operations. The art of posing a problem in terms of $\vec{x}$ and $\bar{z}$ to get these savings requires practice and careful study of individual problems.

The parameter $\rho > 0$ often does not affect whether or not ADMM will eventually converge, but an intelligent choice of ρ can help this technique reach the optimal point faster. Some experimentation can be required, or ρ can be adjusted from iteration to iteration depending on whether the primal or dual variables are converging more quickly [127]. In some cases, ADMM provably converges faster when $\rho \rightarrow \infty$ as the iterations proceed [104].

12.4 GLOBAL OPTIMIZATION

Nonlinear least-squares, IRLS, and alternation are lightweight approaches for nonlinear objectives that can be optimized quickly after simplification. On the other side of the spectrum, some minimization problems not only do not readily admit fast specialized algorithms but also are failure modes for Newton's method and other generic solvers. Convergence guarantees for Newton's method and other algorithms based on the Taylor approximation assume that we have a strong initial guess of the minimum that we wish to refine. When we lack such an initial guess or a simplifying assumption like convexity, we must solve a *global optimization* problem searching over the entire space of feasible output.

As discussed briefly in §9.2, global optimization is a challenging, nearly ill-posed problem. For example, in the unconstrained case it is difficult to know whether $\vec{x}^*$ yields the minimum possible $f(\vec{x})$ *anywhere*, since this is a statement over an infinitude of points $\vec{x}$. Hence, global optimization methods use one or more strategies to improve the odds of finding a minimum:

- Initially approximate the objective $f(\vec{x})$ with an easier function to minimize to get a better starting point for the original problem.